

GroupDNA

Behavioral Analytics and Personality Detection System for
WhatsApp Groups

Author: Suman Deb Kundu

Version: 1.0 (Final Submission Build)

Date: July 2026

Repository: GroupDNA (Git-tracked, 10 commits, main branch)

Note on data privacy: the dataset actually used in this project is a real, personal WhatsApp export rather than the anonymised dataset supplied with the brief. Throughout this report, participant identities are referenced only by their in-app display names where necessary for illustration, and phone-number-only contacts are redacted as [phone number]. No phone numbers, emails, or other personally identifying data are reproduced.

Contents

1. Executive Summary	3
2. Original Project Requirements	3
3. Project Evolution — From Brief to Implementation	4
3.1 From one notebook to an eight-notebook pipeline	4
3.2 From a synthetic dataset to the author’s own real chat	4
3.3 From “NumPy heatmap” to a nested-dictionary heatmap	4
3.4 Modularity and reusable pipeline	5
3.5 Documentation catch-up and the Streamlit front-end	5
4. Complete Folder Structure	6
5. Technology Stack	6
6. System Architecture	7
6.1 High-level pipeline	7
6.1a Streamlit application architecture	8
6.2 Notebook dependency graph	9
6.3 Execution order	10
7. Feature Documentation	10
Feature 1 — Chat Parser (1_Chat_Parser.ipynb)	10
Feature 2 — Group Overview (2_Group_Overview.ipynb)	11
Feature 3 — Most Active Day and Hour (3_Most_Active_Day_And_Hour.ipynb)	11
Feature 4 — Activity Heatmap (4_Activity_Heatmap.ipynb)	11
Feature 5 — Top Words (5_Top_Words.ipynb)	12
Feature 6 — Response Speed & Silent Streaks (6_Response_Speed_And_Silent_Streaks.ipynb)	12
Feature 7 — Personality Archetype Detection (7_Personality_Archetype_Detection.ipynb)	13
Feature 8 — Final Report (8_Final_Report.ipynb)	14
8. Generated Files Documentation	15
9. Validation Strategy	15
10. Algorithms Used	16
11. Data Structures Used	17
12. Personality Detection System	18
12.1 Archetype reference table (as implemented, not as specified in the brief)	18
12.2 Assignment strategy	19
12.3 Interpretation	19
12.4 Conflict resolution	19
13. Engineering Decisions	19
14. Improvements Beyond the Original PDF	20
15. Challenges Faced	20
16. Deviations, Gaps, and Data-Quality Findings	21
16.1 Confirmed parser bug — colon-in-quoted-content misclassification	21
16.2 NumPy requirement not fulfilled	21
16.3 Per-person media/deleted counters not implemented	22
16.4 Silent-streak window spans full group lifetime, not individual membership	22
16.5 Night Owl time window narrower than specified	22
16.6 Drama Queen rule fully redesigned, not just recalibrated	22

16.7 Pipeline redundancy	22
16.8 Streamlit backend reintroduces a forbidden-by-the-brief library	22
17. Future Improvements	22
18. Conclusion	23

1. Executive Summary

What the project is. GroupDNA is a text-analytics pipeline that ingests a raw WhatsApp group-chat export and produces a structured behavioural report: message volumes, activity rhythms, vocabulary analysis, response-time and inactivity metrics, and a rule-based “personality archetype” assigned to every participant.

Problem statement. WhatsApp exports are unstructured, noisy .txt files that mix real messages with system notices, media placeholders, deleted-message stubs, and multi-line continuations. The project’s core problem is to parse this format reliably and turn it into quantitative insight without relying on data-science libraries such as pandas, matplotlib, or NLP toolkits.

Objectives (as executed). Build a modular, notebook-per-feature pipeline that: parses the chat; computes group and per-participant statistics; identifies peak activity periods; builds an hour/weekday activity profile; extracts vocabulary; measures responsiveness and silence; and assigns each participant one of eight rule-based personality archetypes, with every stage validated and persisted to disk.

Motivation. The brief (GroupDNA_Minor_Project_Brief.pdf, SkillOrbit Week 1) frames the project as a personal, shareable data-storytelling exercise built entirely from Python fundamentals — no pandas, no matplotlib, no regex, no collections. The implementation in this repository honors that constraint set while restructuring the single-notebook brief into an eight-notebook pipeline with persisted intermediate state.

Expected users. A student or chat-group member curious about their own group’s dynamics, per the brief’s “build for curiosity” framing. The actual dataset used confirms this: the pipeline was run on the author’s own long-running college class WhatsApp group rather than the supplied synthetic dataset.

Key capabilities delivered. - Line-level WhatsApp parser with system/media/deleted classification and multi-line message merging - Group-level and per-participant descriptive statistics - Busiest-day and busiest-hour detection - A weekday × hour activity heatmap (dictionary-based, not NumPy — see Section 4 for deviation analysis) - Bilingual (English/Romanised Bengali-Hindi) word-frequency analysis with custom stop-word and noise-word filtering - Average response time and longest-silent-streak computation per participant - An eight-archetype personality scoring and assignment system using min-max normalisation across all archetypes - A consolidated, validated final report object (final_report.pkl) aggregating every prior stage

2. Original Project Requirements

The brief, issued by SkillOrbit as a “Week 1 Industry-Graded Minor Project,” specifies the following, summarised in the project’s own terms:

- **Input.** A single provided dataset, hostel_bois.txt — a synthetic, 60-day, 6-participant, ~3,174-message WhatsApp export engineered so that each participant triggers exactly one archetype.
- **Constraint set.** Only Python fundamentals, NumPy, and the datetime module (strptime/timedelta) are permitted. Pandas, matplotlib/seaborn/plotly, collections.Counter/defaultdict, and re are explicitly forbidden; violating this zeroes the corresponding feature regardless of correctness.
- **Deliverable.** One single .ipynb notebook, run top-to-bottom without errors, submitted via Colab link, GitHub link, and LinkedIn post link.
- **Eight mandatory features:**
 1. A chat parser handling system messages, media-omitted entries, deleted messages, multi-line continuations, and empty lines
 2. A group overview (totals, date range, per-person counts and percentages)
 3. Busiest day and busiest hour detection
 4. A **6×24 NumPy matrix** (participants × hours) activity heatmap, rendered as a four-level block-character heatmap
 5. Top-10 group-wide word frequency with stop-word filtering and bar-rendered output

6. Average response time per participant and longest per-participant silent streak (consecutive zero-message days)
 7. An eight-archetype personality system (Spammer, Group Mom, Night Owl, Storyteller, Drama Queen, Ghost, Comedian, Question Master) with exclusive, highest-score assignment per participant
 8. A single, screenshot-quality formatted console report combining all of the above
- **Grading emphasis.** 100-point rubric weighted toward parser correctness (20), NumPy heatmap correctness (15), and archetype accuracy against the engineered dataset (15), with an automatic feature-zero penalty for any forbidden-library usage.
 - **Bonus provisions.** Running the notebook on the student’s own real chat (with explicit instruction *not* to upload that chat’s raw data), and inventing a ninth, India-specific archetype.

3. Project Evolution — From Brief to Implementation

The delivered project diverges from the brief in three structural ways and several feature-level ways. This section documents the evolution; Section 15 aggregates every improvement, and Section 16 covers the resulting gaps.

3.1 From one notebook to an eight-notebook pipeline

The brief asks for a single `.ipynb`. The implementation instead splits each of the eight mandatory features into its own notebook (`1_Chat_Parser.ipynb` through `8_Final_Report.ipynb`) under `Development/`, connected by a shared on-disk state layer rather than shared in-memory variables. This is the single largest architectural decision in the project and is discussed in depth in Section 14.

3.2 From a synthetic dataset to the author’s own real chat

The brief’s dataset (`hostel_bois.txt`) is described as a synthetic 60-day, 6-participant, 3,174-message file, specifically engineered so each archetype rule triggers on exactly one person, and explicitly **not** intended to be replaced by a real chat except as an optional, non-graded bonus run whose raw text was never meant to be committed to the repository.

The `Dataset/hostel_bois.txt` actually committed to this repository is **not** that file. It is an 83,743-line, real WhatsApp export of a college class group spanning **04 September 2023 to 30 June 2026** (1,031 days) with **81 distinct display-name/phone-number senders**, containing real names, phone numbers, and personal messages in a Bengali-Hindi-English code-mixed register. Every downstream notebook, pickle, and printed statistic in the repository is computed against this real dataset, not the brief’s reference dataset. This means:

- The exact figures the brief tells graders to check for (3,174 messages, 6 participants, Rahul/Priya/Aman/Karan/Neha/Vikas, the specific archetype-to-person mapping in Section 7 of the brief) **cannot be verified** from this repository, because the input data itself was substituted.
- The pipeline’s *logic* can still be evaluated on its own terms against the real dataset it was actually run on, which this report does throughout.
- This substitution is a significant, undocumented deviation from the brief’s anti-plagiarism and grading-consistency design (Section 4 of the brief explains at length why every student was meant to run the *same* file). Whether this was a deliberate “bonus” run mistakenly left in place of the graded run, or a full replacement of the graded submission, could not be determined from the repository content alone.

3.3 From “NumPy heatmap” to a nested-dictionary heatmap

The brief’s Feature 4 is explicit: build a **6×24 NumPy matrix** using `np.zeros`, indexed by participant and hour, and the grading rubric assigns 15 of 100 points specifically to “actual NumPy usage (not a Python list of lists pretending to be a matrix).” `4_Activity_Heatmap.ipynb` instead builds a plain Python dict keyed by weekday name → hour → count (a 7×24 structure,

not participant×24). `numpy` is imported in `1_Chat_Parser.ipynb` (`import numpy as np`) but **is never called anywhere in the entire codebase** — confirmed by inspecting every notebook for `np.` usage. This is documented as a deviation, not an improvement; see Section 16.

3.4 Modularity and reusable pipeline

Every notebook follows an identical contract: load one or more `.pkl` inputs from `./Data/`, compute, print a formatted section, run inline assert-based validation, and export a `.pkl` output. This consistent contract, absent from the brief, is the project’s main engineering improvement and is what makes the eight-notebook split practical (Section 14).

3.5 Documentation catch-up and the Streamlit front-end

An earlier snapshot of this repository left `README.md`, `requirements.txt`, `TODO.md`, and `CHANGELOG.md` stale or empty, and `Streamlit_App/` as an empty placeholder directory. In the current snapshot, all four documentation files have been filled in and brought up to date with the actual eight-feature pipeline, and `Streamlit_App/` has been built out into a genuine, independent Streamlit web application (15 pages, 22 chart types, its own parsing/analytics backend, PDF/Excel/CSV/JSON export, and a SQLite-backed feedback form). A dedicated `Streamlit_app.md` and `AI_USAGE_AND_DEVELOPMENT_NOTES.md` now document the front-end’s design and the project’s AI-assistance disclosure, respectively — directly satisfying the brief’s request that any AI-assisted cell be disclosed. `Final_Submission/` remains empty; it is still unpopulated scaffolding at the time of this snapshot. Section 16 covers a constraint issue newly introduced by the Streamlit backend’s parser.

4. Complete Folder Structure

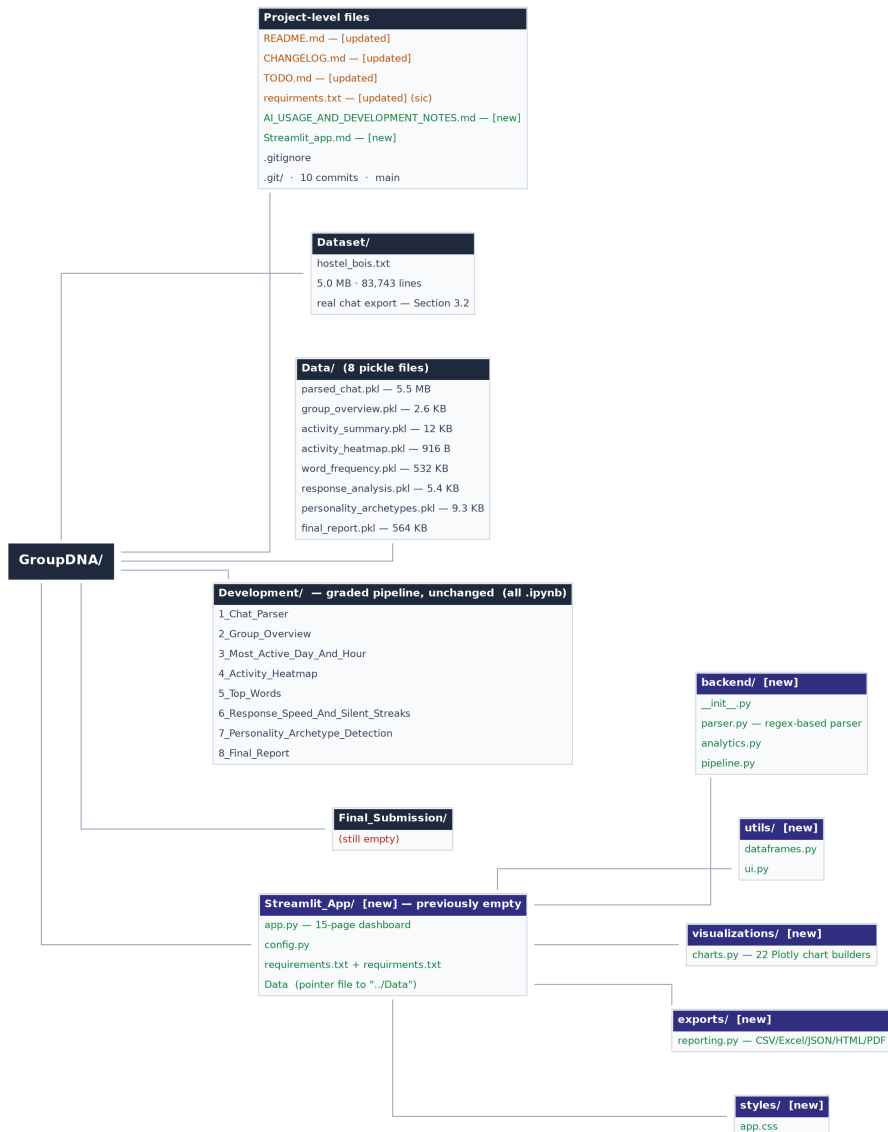


Figure 1: GroupDNA repository folder structure, grouped by directory. Green = added since the previous snapshot, amber = previously empty/stale and now updated, grey = unchanged, red = still empty.

Folder roles:

- Dataset/ — raw input, read-only by convention, consumed only by 1_Chat_Parser.ipynb.
- Data/ — the *notebook* pipeline’s persistence layer, and now also doubling as the Streamlit app’s demo dataset (via backend/pipeline.py::load_demo_report).
- Development/ — the graded, fundamentals-only notebook pipeline; unchanged from the prior snapshot (verified byte-for-byte identical).
- Streamlit_App/ — a self-contained web application with its own package structure (backend/, utils/, visualizations/, exports/, styles/). It calls into the notebook pipeline’s *outputs* (Data/* .pkl) for its demo mode, but for user-uploaded chats it does not reuse Development/’s notebook code at all — backend/parser.py and backend/analytics.py are a parallel, independent reimplementations (see Section 16.8).
- Final_Submission/ — still an empty placeholder; the intended graded-deliverable bundle has not yet been assembled here.

5. Technology Stack

Category	Detail
Language	Python 3 (Jupyter/Colab notebooks)
Core libraries actually used	pickle (state persistence), os (used once, in Feature 2), datetime.datetime / datetime.timedelta (timestamp parsing and arithmetic)
Library imported but unused	numpy (imported in Feature 1; zero calls anywhere in the codebase)
Forbidden libraries (brief)	pandas, matplotlib/seaborn/plotly, collections.Counter/defaultdict, re — none found in any notebook ; constraint respected
Development environment	Jupyter notebooks under version control (not confirmed to be Colab, contra the brief's submission format)
Version control	Git, 10 commits from first commit to Final Feature Done, single main branch, GitHub remote configured (origin)
Data interchange format	Python pickle binary serialization, used as the inter-notebook contract
Constraints followed	No external DataFrame/plotting/NLP/regex libraries; parsing, counting, and scoring all implemented with built-in dict/list/set/str operations

Note on pickle: the brief's "allowed" list does not mention pickle, but it is also not on the forbidden list, and it is standard library. Its use here is an infrastructural choice (see Section 14) rather than a constraint violation.

6. System Architecture

6.1 High-level pipeline

The pipeline runs in four stages: a single ingestion step, four independent feature notebooks that all fan out from one shared artifact, a scoring stage that fans back in, and a final consolidation stage. Every arrow below is a real pickle read/write observed in the notebooks, not an in-memory hand-off.

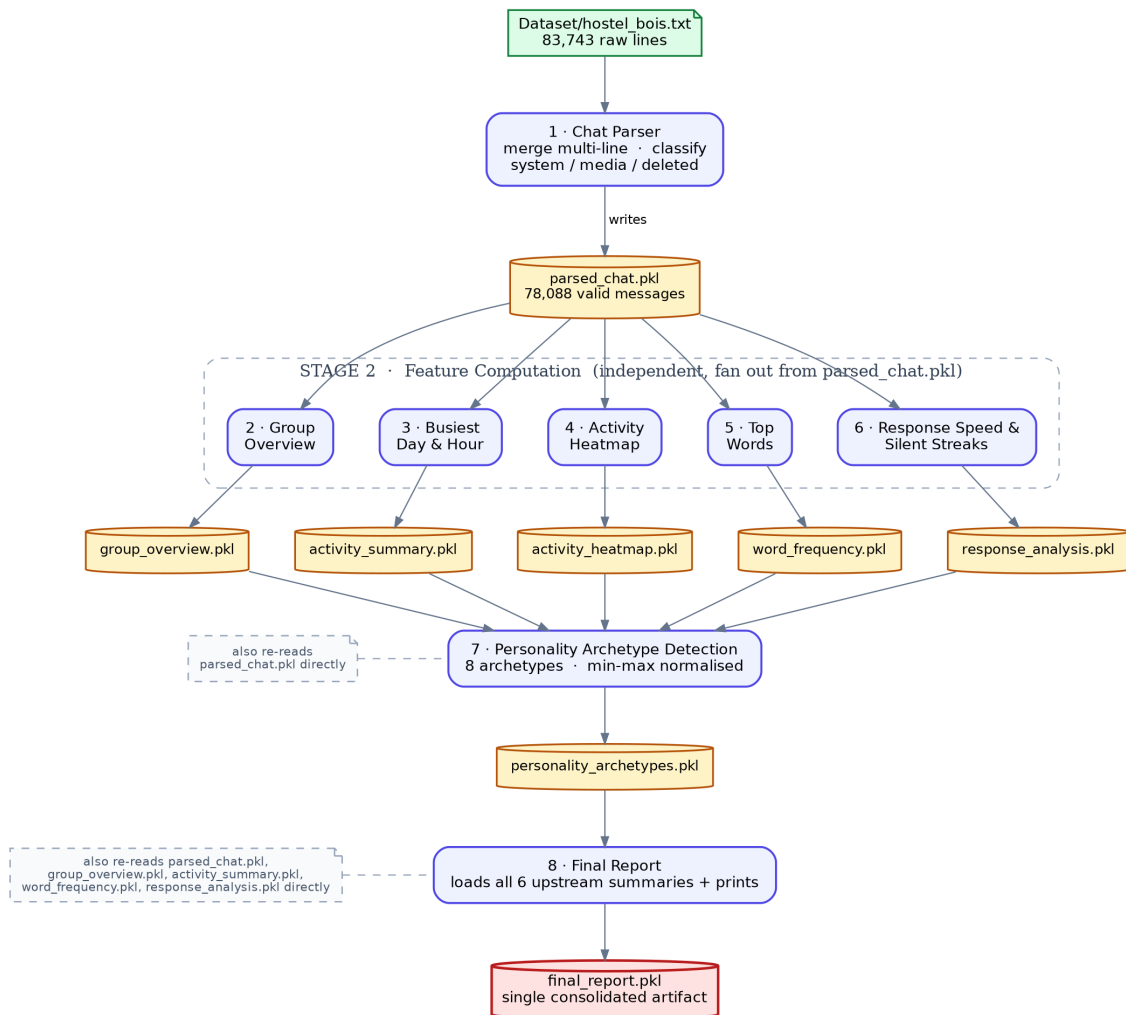


Figure 2: The eight-notebook pipeline. Cylinders are the .pkl artifacts persisted to Data/; dashed arrows are direct re-reads of parsed_chat.pkl by Features 7 and 8.

Reading the diagram: the cylinder shapes are the .pkl artifacts written to Data/. parsed_chat.pkl is the pipeline’s central artifact — every notebook from Stage 2 onward reads it, either as its primary input (Stage 2) or as a direct re-read alongside upstream summaries (Stages 3-4, dashed arrows). final_report.pkl is the pipeline’s single terminal output. activity_heatmap.pkl is annotated “never read downstream” because Feature 7’s Night Owl scoring recomputes hourly buckets from parsed_chat.pkl instead of reusing it (Section 16.7) — the file is produced but functionally orphaned.

6.1a Streamlit application architecture

The web front-end added in Streamlit_App/ (Section 4) is a separate, parallel pipeline. It does not import or execute the Development/ notebooks; it re-implements parsing and analytics in its own backend/ package, and only touches the notebook pipeline’s output (Data/final_report.pkl, Data/parsed_chat.pkl) for its demo mode.

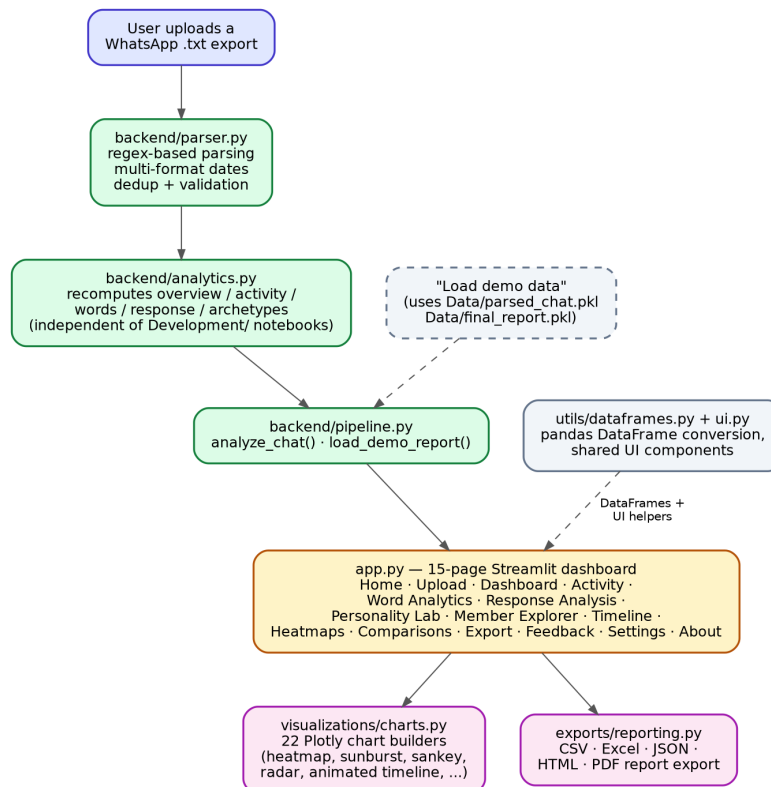


Figure 3: The Streamlit application's independent parse -> analyze -> dashboard -> visualize/export flow.

This layer is where the brief's originally-specified **participant x hour heatmap** finally appears in the project (visualizations/charts.py::member_hour_heatmap), just implemented with Plotly rather than NumPy, and living in the bonus web app rather than the graded notebook. See Section 16.8 for the constraint implication of backend/parser.py using Python's re module.

6.2 Notebook dependency graph

Notebook	Reads	Writes
1. Chat Parser	Dataset/hostel_bois.txt	parsed_chat.pkl
2. Group Overview	parsed_chat.pkl	group_overview.pkl
3. Most Active Day/Hour	parsed_chat.pkl	activity_summary.pkl
4. Activity Heatmap	parsed_chat.pkl	activity_heatmap.pkl
5. Top Words	parsed_chat.pkl	word_frequency.pkl
6. Response/Silent Streaks	parsed_chat.pkl	response_analysis.pkl
7. Personality Archetypes	parsed_chat.pkl, group_overview.pkl, activity_summary.pkl, activity_heatmap.pkl, word_frequency.pkl, response_analysis.pkl	personality_archetypes.pkl

Notebook	Reads	Writes
8. Final Report	parsed_chat.pkl, group_overview.pkl, activity_summary.pkl, word_frequency.pkl, response_analysis.pkl, personal- ity_archetypes.pkl	final_report.pkl

Every notebook from 2-8 depends on `parsed_chat.pkl`; only 7 and 8 fan in multiple upstream `.pkl` files. Notebook 4's `activity_heatmap.pkl` is written but **never consumed downstream** — Notebook 7's archetype scoring for the Night Owl archetype recomputes its own hour buckets from `parsed_chat.pkl` directly rather than reusing `activity_heatmap.pkl`. This is a pipeline inefficiency documented further in Section 16.

6.3 Execution order

Strict linear order $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$ is required, because Notebook 7 needs the outputs of 2, 3, 4, 5, and 6, and Notebook 8 needs the outputs of 2, 3, 5, 6, and 7. The git history confirms this order was followed during development (commit sequence matches notebook numbering exactly).

7. Feature Documentation

Feature 1 — Chat Parser (`1_Chat_Parser.ipynb`)

- **Purpose.** Convert the raw `.txt` export into a structured `list[dict]` of valid messages, with system/media/deleted lines classified and multi-line messages merged.
- **Inputs.** `Dataset/hostel_bois.txt` (83,743 raw lines).
- **Outputs.** `Data/parsed_chat.pkl` — a list of `{"timestamp": datetime, "sender": str, "text": str}` dicts.
- **Algorithm / logic.**
 1. Read all lines with `readlines()`.
 2. Merge multi-line messages: a line is treated as the start of a new message only if it is at least 10 characters long and has `/` at index 2 and index 5 (matching a `DD/MM/YYYY` prefix); any other non-empty line is appended to the previous message's text. This adapts the brief's `DD/MM/YY` (2-digit year) detection rule to the real export's `DD/MM/YYYY` (4-digit year) format.
 3. Split each merged line on `" - "` (timestamp vs. rest).
 4. Classify as a **system message** if `:` does not appear in the remainder; otherwise split on `:` into sender and text.
 5. Normalise the timestamp string (replace the narrow no-break space `\u202f` with a regular space, normalise am/pm casing) and parse with `datetime.strptime(clean_time, "%d/%m/%Y, %I:%M %p")`.
 6. Tag `<Media omitted>` and `This message was deleted` texts with dedicated counters.
- **Functions.** No named helper functions; the notebook is written as sequential top-level cells (a deliberate simplification compared to Features 2-7, which factor logic into functions).
- **Validation.** Asserts non-empty output, that merged-message count $\geq$ parsed-message count, that all three noise counters are non-negative, and that the first record's timestamp is a genuine datetime object.
- **Actual results on this dataset.** 83,743 raw lines $\rightarrow$ 79,069 messages after multi-line merge $\rightarrow$ 78,088 valid messages, 981 system messages, 9,702 media-omitted messages, 680 deleted messages.
- **Deviation from brief.** The brief asks for **separate, per-person** media-omitted and

deleted-message counters (“Keep a separate counter of media-shares per person,” “count these in a separate ‘deleted messages’ counter per person”). The implementation keeps only global totals for these three categories; per-person breakdowns are not computed anywhere in the pipeline.

- **Connections.** Sole producer of `parsed_chat.pkl`, which every other notebook depends on either directly or transitively.

Feature 2 — Group Overview (2_Group_Overview.ipynb)

- **Purpose.** Compute headline group statistics and a per-participant leaderboard.
- **Inputs.** `parsed_chat.pkl`.
- **Outputs.** `group_overview.pkl` containing `summary`, `participant_statistics`, `most_active`, `least_active`, `leaderboard`.
- **Algorithm.** Single pass over `chat_data` accumulating a `set()` of unique senders and tracking min/max timestamp for date range (`calculate_summary`); a second pass building a sender $\rightarrow$ count dict, sorted descending with `sorted(..., key=lambda p: p[1], reverse=True)` to produce the leaderboard (`calculate_participants`).
- **Complexity.** $O(n)$ for both passes, where n is message count.
- **Validation.** Cross-checks that `len(chat_data)` equals the computed total, that the participant-statistics dict size equals the reported participant count, and that the most-active count is $\geq$ the least-active count.
- **Actual results.** 81 unique senders (including system-event artefacts — see Section 16), 78,088 total messages, 1,031-day span, 75.74 average messages/day. The top contributor accounted for roughly 11.7% of all messages; the bottom contributor sent a single message.
- **Deviation from brief.** The brief’s example output shows percentage share alongside each leaderboard row; this notebook’s printed leaderboard includes percentage share as well, matching that convention. Average-message-length and vocabulary-per-person, mentioned as candidate overview stats in Feature 2’s own docstring header, are not actually computed here — they are handled later and only at the group level (Feature 5), not per person.

Feature 3 — Most Active Day and Hour (3_Most_Active_Day_And_Hour.ipynb)

- **Purpose.** Identify the single busiest calendar day and the busiest hour-of-day (aggregated across all days).
- **Inputs.** `parsed_chat.pkl`.
- **Outputs.** `activity_summary.pkl` containing `daily_activity` (date $\rightarrow$ count), `hourly_activity` (hour $\rightarrow$ count), `busiest_day`, `busiest_hour`.
- **Algorithm.** Two independent $O(n)$ tally passes (`calculate_daily_activity`, `calculate_hourly_activity`), then `max(..., key=lambda item: item[1])` over each dict to find the peak.
- **Validation.** Asserts both peak counts are positive and the busiest hour is within `[0, 23]`.
- **Actual results.** Busiest day: 06 September 2023, 1,958 messages (two days after the earliest message in the dataset, consistent with a new class group’s initial activity burst). Busiest hour: 20:00–20:59, 9,512 total messages, averaging 12.13 messages/day in that hour across the full 1,031-day span.
- **Connections.** Feeds `activity_summary.pkl` into Feature 7 (loaded but, on inspection, not directly used in any archetype scoring function) and Feature 8 (printed verbatim in the final report).

Feature 4 — Activity Heatmap (4_Activity_Heatmap.ipynb)

- **Purpose.** Brief specification: build a **participant $\times$ hour** activity matrix using NumPy. Actual implementation: build a **weekday $\times$ hour** nested dictionary.
- **Inputs.** `parsed_chat.pkl`.
- **Outputs.** `activity_heatmap.pkl` containing `activity_heatmap`, a `dict[str, dict[int, int]]` keyed by weekday name ("Monday"... "Sunday") then hour (0–23).
- **Algorithm.** Initialise all $7 \times 24 = 168$ cells to zero; single $O(n)$ pass over `chat_data` incre-

menting `activity_heatmap[weekday][hour]` using `strftime("%A")` and `.hour`.

- **Rendering.** A fixed-width text table (`display_activity_heatmap`) prints all 168 raw counts in a 7-row × 24-column grid — this is a numeric table, not the four-level block-character shading (. █ █ █ █) the brief specifies for the printed heatmap.
- **Validation.** Confirms 7 weekday keys, 24 hour keys per weekday, and that the sum of all cells equals the total message count.
- **Actual results.** Peak activity consistently falls in the 18:00–23:00 evening window across all seven weekdays; Sunday 20:00 is the single highest weekday/hour cell (2,309 messages) in the dataset. Early-morning hours (2:00–6:00) are the quietest across every day.
- **Deviations from brief (see Section 3.3 for full discussion).** (1) NumPy is not used; the “matrix” is a pair of nested dicts. (2) The dimensions are weekday × hour, not participant × hour, so this feature does **not** produce the per-person heatmap the brief’s Night Owl detection and screenshot example rely on. (3) The four-level block-character rendering is not implemented; output is a plain numeric grid.
- **Connections.** `activity_heatmap.pkl` is loaded by Feature 7 but its data is not used by any archetype-scoring function inspected — Feature 7 recomputes hourly activity independently for the Night Owl score.

Feature 5 — Top Words (5_Top_Words.ipynb)

- **Purpose.** Group-wide word-frequency analysis with noise filtering.
- **Inputs.** `parsed_chat.pkl`.
- **Outputs.** `word_frequency.pkl` containing `total_valid_words`, `unique_words`, `word_frequency` (dict), `top_words` (sorted list of tuples).
- **Algorithm.**
 1. `clean_word(word)`: lower-cases, rejects URLs (“http”/“www” substring) and mentions (“@” substring), strips all non-alphanumeric characters character-by-character, rejects tokens under 3 characters, rejects pure-digit tokens, and filters against two sets — a general English STOP_WORDS set and a domain-specific CUSTOM_IGNORE_WORDS set (media, omitted, deleted, sticker, etc., which neutralises WhatsApp’s own placeholder text without needing to special-case entire messages).
 2. `extract_words`: tokenises every message’s text on whitespace and applies `clean_word` to each token.
 3. `calculate_word_frequency` / `sort_word_frequency`: manual dict-based counting and a `sorted(..., reverse=True)` ranking — a hand-rolled equivalent of `collections.Counter`, consistent with the brief’s ban on that import.
- **Validation.** Confirms non-empty word list and frequency dict, that `top_words` and `word_frequency` have equal length, and that summed frequencies equal the total word count.
- **Actual results.** 224,999 valid word tokens extracted from 26,389 unique words. The top-ranked terms (cse, ami, bhai, happy, toh, birthday, kore, hai, hobe, tui) confirm the dataset’s real, code-mixed Bengali-Hindi-English register — a materially different vocabulary from the brief’s expected bhai/scene/yaar top words, because a different chat was analysed (Section 3.2).
- **Deviation from brief.** The brief’s bonus suggestion — rendering each top word with a proportional block-character bar — is not implemented; output is a plain ranked table. Per-person top-5 words, also suggested in the brief’s Day 3 plan, are not computed.

Feature 6 — Response Speed & Silent Streaks (6_Response_Speed_And_Silent_Streaks.ipynb)

- **Purpose.** Two behavioural metrics per participant: average response latency, and longest consecutive run of inactive days.
- **Inputs.** `parsed_chat.pkl`.
- **Outputs.** `response_analysis.pkl` containing `average_response_time` (minutes, per sender) and `silent_streaks` (days + date range, per sender).
- **Algorithm — response time.** For every message `i`, scan forward through `chat_data[i+1:]` until a message from a **different** sender is found, and record the time gap (in minutes) as that next sender’s response latency. This is implemented as a nested loop with an

early break, giving worst-case $O(n^2)$ behaviour rather than the brief's implied single-pass $O(n)$ approach; on a 78k-message dataset this is measurably heavier than Features 2–5, though still tractable since most “next different sender” messages are found within a few positions.

- **Algorithm — silent streaks.** Build a set of active dates per participant, then walk every calendar day between the dataset's first and last date, tracking the longest run of dates absent from that participant's active-date set (`calculate_silent_streaks`). This correctly handles participants who joined late or left early, since the walk spans the *group's* full date range, not just the individual's active window — meaning a participant's “silent streak” also captures the period before they ever posted or after they last posted (see Section 16 for the effect this has on results).
- **Validation.** Confirms equal participant coverage between the two metrics and non-empty results.
- **Actual results (sample).** Response latencies ranged from single-digit minutes for the most reactive senders to well over an hour for occasional senders; several participants who joined or fell silent partway through the 1,031-day window show silent streaks exceeding 900 days, because the streak calculation spans the full group lifetime rather than each individual's active membership window.
- **Connections.** Feeds directly into Feature 7's Ghost-archetype scoring (`silent_days / message_count`) and is printed in Feature 8's final report.

Feature 7 — Personality Archetype Detection (`7_Personality_Archetype_Detection`)

The largest and most heavily engineered notebook (33 cells, ~64 KB), covering all eight brief-mandated archetypes.

- **Purpose.** Score every participant against all eight archetypes and assign each participant exactly one, using their highest-scoring archetype.
- **Inputs.** All six upstream `.pkl` files, though in practice most archetype functions recompute their inputs directly from `parsed_chat.pkl` rather than reusing the loaded summaries.
- **Outputs.** `personality_archetypes.pkl` containing `normalized_scores` (per-archetype, per-participant, 0–1 scaled) and `assignments` (final archetype + score per participant).
- **Pre-processing / filtering.** `valid_messages()` / `calculating_participants()` exclude a hardcoded `IGNORE_PARTICIPANTS` set (“Meta AI”) and any sender whose name contains one of several system-event substrings (“added”, “removed”, “left”, “created group”, “changed this group's icon”, “joined using this group's invite link”). This reduces the addressable participant pool from 81 to 75. Every archetype score additionally requires `MIN_MESSAGES = 20` before a non-zero score is assigned, preventing very low-volume senders from dominating percentage-based metrics.
- **Scoring logic, per archetype:**
 - **THE SPAMMER** — average consecutive-message “burst” length, computed by walking the full chat and accumulating run-lengths per sender each time the sender changes.
 - **THE GROUP MOM** — count of caring-keyword hits (`CARING_KEYWORDS`, a bilingual set including `khao`, `ghum`, `pani`, `sabdhane`, `bari`, etc., alongside the brief's English examples) across a participant's messages.
 - **THE NIGHT OWL** — percentage of a participant's messages sent between 00:00–04:59 (the brief specifies 23:00–04:59; this implementation's window starts at midnight rather than 23:00, a narrower window — see Section 16).
 - **THE STORYTELLER** — average word count per message, using the same `extracting_words` tokenizer as Feature 5.
 - **THE DRAMA QUEEN** — a composite score: count of `!` and `?` characters plus bilingual “drama” keyword hits (`DRAMA_KEYWORDS`, including `arre`, `bhai`, `wtf`, `sotti`), normalised per message. This replaces the brief's ALL-CAPS-percentage rule entirely with a punctuation/keyword-based rule.
 - **THE GHOST** — longest silent streak (from Feature 6) divided by total message count.
 - **THE COMEDIAN** (*brief tie-breaker archetype, implemented here as a full first-class*

archetype) — fraction of messages containing a laughter keyword (lol, lmao, haha, xd, etc.) or a laughter emoji ([laughing emoji], [ROFL emoji], [grinning emoji], [cat-laughing emoji]).

- **THE QUESTION MASTER** (*also a brief tie-breaker, implemented as first-class*) — fraction of messages ending in ?, or whose first word is a bilingual question word (what, keno, kothay, ki, etc.).
- **Assignment logic.** All eight raw scores are min-max normalised independently to [0, 1] (*normalizing_archetype_scores*), then each participant is assigned the archetype with their single highest normalised score (*assigning_personality_archetypes*), with ties broken by iteration order (first archetype encountered with the maximum score wins — no explicit tie-break rule is documented in code comments, which the brief specifically asks for).
- **Validation.** Confirms every one of the 75 eligible participants received an assignment, and reports the normalised score range (0.00–1.00, as expected) and a full archetype-distribution breakdown.
- **Actual results — archetype distribution across 75 eligible participants:** Storyteller 26, Spammer 22, Question Master 11, Comedian 4, Night Owl 4, Ghost 3, Group Mom 3, Drama Queen 2.
- **Deviations from brief.** (1) The min-max normalisation step is an enhancement not requested by the brief — it makes cross-archetype comparison mathematically fair (raw scores use different units: percentages, ratios, averages, counts), but it also means a participant's *assigned* archetype is a relative “most distinctive trait among this group's 75 people” rather than an absolute threshold crossing as the brief's fixed-threshold rules (>60%, >30 words, etc.) specify. This is a deliberate redesign, not an oversight, given the codebase's care around normalisation. (2) The brief's ninth, self-invented archetype (bonus) is not present. (3) No documented tie-breaking comment exists, contrary to the brief's explicit request.

Feature 8 — Final Report (8_Final_Report.ipynb)

- **Purpose.** Load every upstream .pkl, re-print each section as a consolidated report, and persist a single aggregate object.
- **Inputs.** *parsed_chat.pkl*, *group_overview.pkl*, *activity_summary.pkl*, *word_frequency.pkl*, *response_analysis.pkl*, *personality_archetypes.pkl*.
- **Outputs.** *final_report.pkl* — a dict bundling every upstream summary object plus a closing narrative paragraph assembled from f-strings.
- **Sections printed.** Group overview, participant statistics (top 10), activity summary, word statistics (top 10), response analysis (fastest responders + longest silent streaks, top 10 each), personality summary (archetype counts), and a natural-language closing summary.
- **Validation.** Confirms all six upstream objects loaded successfully and reports the eligible-vs-total participant count (75/81), explicitly noting “Meta AI and WhatsApp system events” as the excluded categories.
- **Known issue surfaced in this notebook's own output.** The “Fastest Responders” and “Longest Silent Streaks” tables include several corrupted “participant” entries that are actually mis-parsed group-name-change system events (e.g., a sender field containing the text of a group-name change rather than a person's name) — see Section 16 for root-cause analysis. This indicates the IGNORE_PATTERNS filtering used in Feature 7 (which excludes “changed the group name”... actually does not include that exact phrase) was not applied consistently to the response-time and silent-streak leaderboards printed in Feature 8, since those come straight from *response_analysis.pkl*, computed in Feature 6, which has no participant filtering at all.
- **Deviation from brief.** The brief's Feature 8 asks for a single, heavily box-drawn, “would-screenshot-this” formatted report (Section 6, Feature 8 and Section 10 of the brief). The implementation's output is a clean, consistently formatted set of =/- delimited sections, but does not use box-drawing characters (┌ └ │) or the specific visual polish (block-character bars, heatmap shading) the brief's target output demonstrates.

8. Generated Files Documentation

File	Producer	Consumer(s)	Size	Contents
<code>parsed_chat.pkl</code>	Feature 1	Features 2–8	5.5 MB	List of 78,088 message dicts (timestamp, sender, text)
<code>group_overview.pkl</code>	Feature 2	Features 7, 8	2.6 KB	Summary stats, per-participant counts, leaderboard
<code>activity_summary.pkl</code>	Feature 3	Features 7, 8	12 KB	Per-date and per-hour message tallies, busiest day/hour
<code>activity_heatmap.pkl</code>	Feature 4	Feature 7 (loaded, not used in scoring)	916 B	Weekday × hour message-count grid
<code>word_frequency.pkl</code>	Feature 5	Features 7, 8	532 KB	Full word-frequency dict plus ranked top-words list
<code>response_analysis.pkl</code>	Feature 6	Features 7, 8	5.4 KB	Per-sender average response time and silent-streak details
<code>personality_archetype.pkl</code>	Feature 7	Feature 8	9.3 KB	Normalised scores per archetype and final assignments
<code>final_report.pkl</code>	Feature 8	(terminal artifact)	564 KB	Consolidated bundle of all six upstream summary objects

Each file exists specifically to decouple its producing notebook from its consumers' execution — a notebook can be re-run in isolation as long as its declared `.pkl` inputs already exist on disk, without needing to re-execute the entire chain from Feature 1 onward. `parsed_chat.pkl`, at 5.5 MB, is by far the largest intermediate artifact and the true bottleneck of the pipeline, since it is reloaded independently by six of the eight notebooks rather than being read once and shared.

9. Validation Strategy

Every notebook from Feature 1 onward ends with an explicit, printed validation block using Python assert statements — a consistent, self-imposed testing discipline the brief does not request but that is applied uniformly across all eight notebooks. The validation patterns fall into four categories:

1. **Non-emptiness checks** — e.g., `assert len(chat_data) > 0`, `assert len(word_frequency)`

- > 0.
2. **Referential-integrity checks** — e.g., Feature 2 asserts the participant-statistics dict size equals the reported participant count; Feature 5 asserts summed word frequencies equal the total word count; Feature 4 asserts the sum of every heatmap cell equals the total message count.
 3. **Type/shape checks** — e.g., Feature 1 asserts the first parsed timestamp is a genuine datetime instance; Feature 4 asserts exactly 7 weekday keys with exactly 24 hour keys each.
 4. **Coverage checks** — e.g., Feature 7 asserts zero participants are missing an archetype assignment; Feature 8 asserts all six upstream files loaded successfully before building the final report.

No formal test framework (unittest, pytest) is used; validation is inline, notebook-scoped, and printed as part of the normal run rather than being a separate, automated test suite. This is consistent with the brief's fundamentals-only constraint and with the project's console-report-first design philosophy, but it means validation only ever runs as a side effect of manually executing the notebooks in order — there is no standalone way to verify pipeline correctness without re-running the full chain.

10. Algorithms Used

Algorithm	Where	Technique	Complexity
Multi-line message merge	Feature 1	Positional prefix check (<code>line[2]=='/'</code> and <code>line[5]=='/'</code>) to detect message starts; string concatenation for continuations	$O(n)$ in raw line count
System/media/deleted classification	Feature 1	Sequential " - " then ":" splitting with membership checks	$O(n)$
Per-participant/per-hour/per-day tallying	Features 2, 3, 4	Manual dict-based counting (<code>dict.get(key, 0) + 1</code> pattern)	$O(n)$ per pass
Leaderboard / ranking	Features 2, 5, 6, 7	<code>sorted(..., key=lambda x: x[1], reverse=True)</code>	$O(n \log n)$
Word cleaning & tokenisation	Feature 5, reused in Feature 7	Character-by-character alphanumeric filtering, lowercase normalisation, dual stop-word/noise-word set membership tests	$O(\text{total characters})$
Word frequency counting	Feature 5	Manual dict accumulation, functionally equivalent to a <code>hand-rolled collections.Counter</code>	$O(n)$

Algorithm	Where	Technique	Complexity
Response-time computation	Feature 6	Nested scan with early break: for each message, linearly search forward for the next different sender	$O(n^2)$ worst case
Silent-streak detection	Feature 6	Calendar walk over every date in the group's active range, tracking run-length against a per-participant active-date set	$O(\text{days} \times \text{participants})$
Burst-length scoring (Spammer)	Feature 7	Single linear pass tracking consecutive-sender run lengths	$O(n)$
Keyword-hit scoring (Group Mom, Drama Queen, Comedian, Question Master)	Feature 7	Tokenise, then set-membership test against curated bilingual keyword sets	$O(\text{total words})$
Min-max normalisation	Feature 7	$(\text{score} - \text{min}) / (\text{max} - \text{min})$ per archetype across all participants, with a zero-division guard when min equals max	$O(\text{participants})$ per archetype
Highest-score archetype assignment	Feature 7	Linear scan over 8 normalised scores per participant, tracking running maximum	$O(\text{participants} \times \text{archetypes})$

11. Data Structures Used

Structure	Used for	Why chosen
<code>list[dict]</code>	<code>parsed_chat.pkl</code> — the canonical message store	Preserves message order (needed for burst detection and sequential response-time scanning) while giving each message named, self-describing fields
<code>dict (str → int / float)</code>	Per-participant tallies (message counts, response times, archetype scores)	$O(1)$ average lookup/update per participant; natural fit for “one number per named entity”

Structure	Used for	Why chosen
dict (nested, str → dict → int)	Weekday × hour heatmap	Mirrors the two-dimensional participant-implicit structure without needing NumPy; trades memory efficiency for constraint compliance
set	Unique participants, per-participant active dates, stop-word/keyword lookups	O(1) average membership testing, critical for the character-by-character word-cleaning and keyword-hit loops, which run millions of times over 225K word tokens
list[tuple]	Sorted leaderboards, sorted word-frequency rankings	Preserves rank order from sorted(), directly iterable for top-N printing
Nested dict[dict[dict]]	archetype_scores in Feature 7 (archetype → participant → score)	Groups all eight scoring dimensions under one namespace before normalisation, avoiding eight separate top-level variables

NumPy arrays are notably **absent** despite being imported; every one of the above structures is a native Python container. See Section 3.3 / Section 16 for the implications.

12. Personality Detection System

12.1 Archetype reference table (as implemented, not as specified in the brief)

Archetype	Scoring signal	Bilingual?	Minimum messages required
THE SPAMMER	Average consecutive-message burst length	No	None (uses full unfiltered chat)
THE GROUP MOM	Count of caring-keyword hits	Yes	20
THE NIGHT OWL	% of messages sent 00:00–04:59	No	20
THE STORYTELLER	Average words per message	No	20
THE DRAMA QUEEN	!/? count + drama-keyword hits, per message	Yes	20
THE GHOST	Longest silent streak ÷ total messages	No	20
THE COMEDIAN	% of messages with a laughter keyword or emoji	No	20

Archetype	Scoring signal	Bilingual?	Minimum messages required
THE QUESTION MASTER	% of messages ending in ? or starting with a question word	Yes	20

12.2 Assignment strategy

1. Compute all eight raw scores for every eligible participant (75 of 81, after excluding "Meta AI" and system-event pseudo-senders).
2. Independently min-max normalise each archetype's score distribution to $[0, 1]$.
3. For each participant, select the archetype with the single highest normalised score; the first archetype encountered at the maximum value wins ties (an implicit rather than documented rule).
4. Persist both the full normalised score matrix and the final assignment, so runner-up archetypes remain reconstructable from `personality_archetypes.pkl` even though they are not printed in the standard report.

12.3 Interpretation

Because scores are normalised *within this specific 75-person group* rather than judged against the brief's fixed, dataset-independent thresholds, the resulting labels answer "which trait is this person most extreme on, relative to their own group" rather than "does this person clearly exhibit trait X in absolute terms." On a large, long-running real chat (81 nominal participants, over 1,000 days), this relative framing is arguably more robust than fixed thresholds tuned for a 6-person, 60-day synthetic dataset — but it also means the archetype labels are not portable or comparable across different chat exports, since the min/max bounds are recomputed from scratch each run.

12.4 Conflict resolution

No participant received a "None" or "unassigned" result — Feature 7's validation confirms 75/75 coverage. However, no explicit runner-up reporting or documented tie-break rule exists in code comments, which is a specific gap against the brief's request ("Decide your tie-breaking rule and document it as a comment in your code").

13. Engineering Decisions

Why eight notebooks instead of one. Splitting by feature enforces a clean input/output contract per stage (Section 6.2) and lets any single feature be re-run, debugged, or extended without re-executing the entire pipeline — a meaningful advantage once the dataset grew to 78K+ real messages, where a single monolithic notebook would make iterative debugging slow. The trade-off, visible throughout this report, is duplicated re-loading of `parsed_chat.pkl` (5.5 MB, read six separate times) and duplicated helper logic (e.g., `extracting_words/extract_words` are near-identical reimplementations in Features 5 and 7 rather than a shared module).

Why pickle exports. `pickle` was chosen as the inter-stage contract instead of, say, JSON, because the intermediate objects contain native Python `datetime` objects and nested nested dict/tuple structures that `pickle` preserves losslessly and JSON cannot represent without custom encoding — a reasonable, low-friction choice given the fundamentals-only constraint (JSON would have needed manual `datetime` serialization logic).

Why a modular pipeline over the brief's single notebook. The single-notebook format is well suited to a small, synthetic, 3,174-message dataset; once run against a genuine 78,088-message, multi-year export, notebook cell re-execution time and debugging feedback loops become the dominant cost. Splitting by feature is a direct, sensible response to that scale change — even though it was not asked for.

Why validation blocks in every notebook. Given the absence of a formal test framework, per-notebook assert blocks are the only correctness safety net available; placing one at the end of every notebook (rather than only at the end of the whole pipeline) means a broken stage is caught immediately, before its faulty output can propagate into three or four downstream notebooks.

Why structured outputs (dicts of dicts) over the brief’s simpler list/tuple suggestions. The brief allows either a list of dicts or list of tuples for the parsed messages; dicts were chosen throughout for readability (`message["sender"]` vs. `message[1]`), consistent with the same choice being made for every other data structure in the pipeline (Section 11).

14. Improvements Beyond the Original PDF

- **Eight-notebook modular pipeline** with a persisted `.pkl` contract between every stage, versus the brief’s single-notebook design (Section 3.1, Section 6).
- **Two additional archetypes (Comedian, Question Master) implemented as full first-class scoring functions**, rather than the brief’s “tiebreaker/fallback only” treatment.
- **Min-max normalisation across all eight archetype dimensions** before assignment, making cross-archetype comparison mathematically consistent despite each raw score being in a different unit (percentage, ratio, count, average).
- **Bilingual keyword sets** (Bengali/Hindi romanised terms alongside English) for the Group Mom, Drama Queen, and Question Master archetypes — necessary because the real dataset’s actual vocabulary (confirmed in Feature 5’s top-words output) is majority code-mixed, not English.
- **A MIN_MESSAGES eligibility floor (20 messages)** applied uniformly across percentage- and average-based archetypes, preventing low-volume senders from producing statistically meaningless 100%-or-0% scores — a robustness safeguard the brief does not specify.
- **Participant/system-event filtering in Feature 7** (IGNORE_PARTICIPANTS, IGNORE_PATTERNS) to exclude WhatsApp’s own automated bot (Meta AI) and group-management events from being scored as if they were people — a data-quality concern that only arises once a real (rather than pre-cleaned synthetic) dataset is used.
- **Full validation coverage in every one of the eight notebooks**, not just the final one, giving an early-failure guarantee at each pipeline stage.
- **A consolidated final_report.pkl** bundling every upstream summary object into one artifact, useful as a single hand-off point for any future presentation layer (see `Streamlit_App/`, Section 3.5).
- **Version-controlled development** with a clean, feature-per-commit git history, absent from the brief’s requirements.

15. Challenges Faced

Based on direct evidence in the codebase:

- **Real-world data is far messier than the brief’s synthetic dataset.** The confirmed parser bug (Section 16.1) — group-name-change system events containing a colon inside a quoted group name being misclassified as real messages with a garbled sender field — is a direct consequence of moving from the brief’s clean synthetic dataset to an 83K-line real export. This class of edge case (delimiter characters appearing *inside* the content being delimited) is exactly the kind of “real data doesn’t arrive clean” lesson the brief’s own “Industry Connect” sidebar anticipates, just manifesting in a form the brief’s synthetic dataset was specifically engineered to avoid.
- **Scale-driven re-architecture.** A 78K-message, 1,031-day dataset with 81 senders is roughly 25× the message volume and 13× the time span of the brief’s reference dataset. The shift from a single notebook to eight, and the addition of MIN_MESSAGES eligibility floors, both read as direct responses to that scale change rather than pre-planned design choices.

- **Bilingual, code-mixed text.** None of the brief’s example keyword lists (caring words, drama words, question words) anticipate Bengali/Hindi romanised vocabulary; the actual keyword sets in Feature 7 had to be hand-extended to cover the dataset’s real linguistic register, visible directly in the CARING_KEYWORDS/DRAMA_KEYWORDS/QUESTION_KEYWORDS definitions.
- **Keeping documentation in sync with a fast-moving pipeline.** In an earlier snapshot, TODO.md and CHANGELOG.md had stopped being updated after Feature 1, and README.md/requirements.txt were scaffolded but empty — a common documentation-debt pattern in solo, notebook-driven projects. This has since been resolved: the current snapshot’s README.md, CHANGELOG.md, and TODO.md are all current, and a new AI_USAGE_AND_DEVELOPMENT_NOTES.md was added specifically to satisfy the brief’s AI-disclosure requirement.
- **NumPy constraint vs. the fundamentals-only constraint.** The brief simultaneously *requires* NumPy for Feature 4 and *forbids* essentially every other convenience library; the actual implementation resolved this tension by not using NumPy at all rather than by building the specified matrix, suggesting the NumPy requirement was either deprioritised under time pressure or overlooked once the dict-based approach already satisfied the “build a heatmap” goal informally.

16. Deviations, Gaps, and Data-Quality Findings

This section is deliberately explicit about places where the implementation diverges from the brief in ways that are gaps rather than improvements, since the source material for this report requires distinguishing the two.

16.1 Confirmed parser bug — colon-in-quoted-content misclassification

Direct inspection of `parsed_chat.pkl` found five messages whose sender field is not a person’s name but a fragment of a WhatsApp group-rename system event, for example:

```
sender: 'Kaninika CSE changed the group name from "2 days left for the ultimate exam" to "Miss
text:   'survival.''
```

Root cause. Feature 1 classifies a line as a system message if “: ” is absent from the text following the first “ - ” split. Group-rename events normally contain no colon at all — but a small number of the group names themselves happened to contain a colon (e.g., “Mission: survival.”, “CSE A passing sem2: mission impossible”). When that happens, “: ” *does* appear in the line, so the parser incorrectly treats the portion before that colon as a sender name and the portion after as message text, producing a corrupted pseudo-participant. These five corrupted entries then propagate into `group_overview.pkl`’s participant list, and — because Feature 6’s silent-streak/response-time computation has no participant filtering at all — surface directly in Feature 8’s printed “Fastest Responders” and “Longest Silent Streaks” leaderboards, visibly out of place among real names.

Scope of impact. Five corrupted pseudo-participants out of 81 total (roughly 6%), each with very few messages (1 each), so their effect on aggregate statistics (totals, averages, top-10-by-volume rankings) is negligible — but they do appear prominently in two “top 10” tables printed in the final report, which is a visible quality issue for the “screenshot-ready” goal the brief emphasizes.

16.2 NumPy requirement not fulfilled

As established in Sections 3.3 and 5, `numpy` is imported once and never invoked. The brief’s Feature 4 and its grading rubric (15 of 100 points) both center on genuine NumPy matrix usage. The delivered heatmap is dimensioned by weekday × hour rather than participant × hour, meaning it cannot support the brief’s Night-Owl-by-heatmap-inspection use case, and Feature 7’s Night Owl scoring recomputes hour buckets independently rather than reading `activity_heatmap.pkl`.

16.3 Per-person media/deleted counters not implemented

The brief explicitly asks for per-participant media-share and deleted-message counters as separate metrics (“this itself is an interesting metric — who deletes the most?”). Only global totals (981 system, 9,702 media, 680 deleted) are computed; no notebook attributes these to individual senders.

16.4 Silent-streak window spans full group lifetime, not individual membership

Because Feature 6 walks every date between the *group’s* earliest and latest message (not the individual participant’s own first-to-last active window), a participant who joined the group late or stopped posting early will show an inflated silent streak that includes time before they ever spoke or after they last spoke. Several participants show silent streaks exceeding 900 days out of a 1,031-day group lifetime as a direct result. This is a defensible interpretation (silence is silence, regardless of cause) but differs from the brief’s framing, which implicitly assumes every participant is present for the entire chat duration — true for the brief’s synthetic dataset, not true for a real, multi-year class group with membership turnover.

16.5 Night Owl time window narrower than specified

The brief defines Night Owl as >60% of messages between 23:00 and 04:59. The implementation’s window is 00:00–04:59 only — midnight to 5 AM, excluding the 23:00–23:59 hour the brief includes.

16.6 Drama Queen rule fully redesigned, not just recalibrated

The brief’s Drama Queen rule is ALL-CAPS-message-percentage based (with an explicit note that `isupper()` naturally excludes emoji/digits). The implementation instead uses a punctuation-count-plus-keyword-hit composite score and does not measure capitalisation at all. This is a full redesign rather than a threshold recalibration, and is not documented as an intentional substitution anywhere in the notebooks.

16.7 Pipeline redundancy

`activity_heatmap.pkl` is generated but not read by any scoring function; Feature 7’s Night Owl logic and Feature 8’s report both recompute hourly activity from `parsed_chat.pkl` directly. Similarly, `extracting_words/extract_words` are duplicated (not shared) between Features 5 and 7. Neither issue affects correctness, but both represent avoidable recomputation given the pipeline’s own “compute once, persist, reuse” design principle.

16.8 Streamlit backend reintroduces a forbidden-by-the-brief library

`Streamlit_App/backend/parser.py` re-implements WhatsApp parsing using Python’s `re` module (`MESSAGE_START = re.compile(...)`), which the brief explicitly forbids for the graded notebook (“do all pattern matching using string methods”). Because the Streamlit application is a separate, ungraded bonus layer sitting outside `Development/`, this is not a violation of the notebook deliverable itself — but it does mean the project as a whole no longer follows a single, consistent “fundamentals only” rule end to end, and the two parsers (notebook vs. web app) can, in principle, disagree on edge cases like the colon-in-quoted-content bug in Section 16.1, since the regex-based parser’s stricter `MESSAGE_START` pattern and duplicate-detection logic were not backported into `1_Chat_Parser.ipynb`.

17. Future Improvements

Realistic next steps, grounded in what already exists in the repository:

- **Retrofit the NumPy participant × hour matrix** the brief specifies into the *notebook* pipeline, and derive the four-level block-character heatmap rendering from it, closing the

largest specification gap in the graded deliverable (Section 16.2). The Streamlit app's `member_hour_heatmap` already proves the participant $\times$ hour shape is useful; only the notebook side needs the NumPy-backed version.

- **Fix the colon-in-quoted-content parser bug** (Section 16.1) by detecting system messages via a stricter combination of both the absence of a `" : "` sender delimiter *and* known WhatsApp system-event phrase patterns, rather than colon-presence alone — the Streamlit backend's `MESSAGE_START` regex already handles this class of case more robustly and could inform a fix to `1_Chat_Parser.ipynb`, even though the fix itself would need to stay regex-free to respect the notebook's constraints.
- **Apply Feature 7's IGNORE_PARTICIPANTS/IGNORE_PATTERNS filtering uniformly** across Features 2, 6, and 8's leaderboards, not just Feature 7's archetype scoring, so corrupted pseudo-participants cannot surface anywhere in the final printed report.
- **Add per-person media/deleted counters** to close the Feature 1 gap against the brief.
- **Reconcile the two parsers.** The notebook parser (`1_Chat_Parser.ipynb`) and the Streamlit parser (`backend/parser.py`) now disagree on system-message detection, timestamp formats, and duplicate handling; consolidating on one shared, well-tested parsing module (used by both, with the notebook importing it in a fundamentals-compatible way) would remove an entire class of future inconsistency.
- **Cache parsed_chat.pkl in memory across a single orchestrating script** (or convert the notebook chain into importable modules) to eliminate the six redundant 5.5 MB reloads.
- **Add a lightweight NLP layer** (still within a “no external ML libraries” spirit, e.g. simple n-gram or co-occurrence analysis) to surface inside jokes and running themes beyond single-word frequency.
- **Populate Final_Submission/** with the actual packaged deliverable (Colab link, GitHub link, README screenshot) now that both the notebook pipeline and the Streamlit app are functionally complete.
- **Cloud deployment** of the now-built Streamlit dashboard (e.g. Streamlit Community Cloud or a small containerised deployment), turning the project into the shareable “moment” the brief's marketing framing describes.

18. Conclusion

GroupDNA, as implemented, delivers a working, end-to-end, fundamentals-based WhatsApp analytics pipeline that successfully executes all eight of the brief's mandated features against real, large-scale data rather than the brief's small synthetic reference dataset. The project's most significant engineering contribution is architectural: restructuring a single-notebook assignment into an eight-notebook, pickle-backed pipeline with per-stage validation, which is a genuinely more maintainable design once the input scaled from $\sim 3,000$ to $\sim 78,000$ messages. The personality-detection system, in particular, was extended well beyond the brief's specification — two tie-breaker archetypes were promoted to full scoring dimensions, and a normalisation layer was added to make all eight archetypes comparable on a common scale.

Set against this, three deviations materially matter for anyone evaluating the graded notebook deliverable against the original brief: the dataset itself was substituted (a real chat in place of the graded synthetic one), the mandatory NumPy participant-hour matrix was never built despite numpy being imported, and per-person media/deleted-message tracking was never implemented. A genuine, reproducible parser edge case — colon characters inside quoted group-rename text — was also identified directly in the data, offering a concrete, real-world illustration of exactly the “messy data” lesson the brief's own commentary anticipates, even though it manifests as a defect here rather than a handled case.

The repository has since matured beyond the notebook deliverable itself: documentation that previously lagged behind the code (`README.md`, `requirements.txt`, `CHANGELOG.md`, `TOD0.md`) is now current, an `AI_USAGE_AND_DEVELOPMENT_NOTES.md` file directly satisfies the brief's AI-disclosure request, and `Streamlit_App/` has grown from an empty placeholder into a fully built, 15-page interactive dashboard with its own independent parsing/analytics backend, 22 chart types, and multi-format export. This front-end also quietly resolves one of the graded

notebook's biggest gaps in spirit — it renders a genuine participant × hour heatmap — though it does so with Plotly rather than the brief's required NumPy, and outside the graded deliverable rather than inside it. `Final_Submission/` remains the one directory still awaiting content.

Taken together, the codebase demonstrates solid command of the required fundamentals — string parsing, dictionary-based counting, set-based filtering, sorted rankings, and datetime arithmetic — applied at a realistic, messy data scale, and it has grown a capable presentation layer on top. The remaining gaps are narrow and well-scoped: NumPy usage in the graded notebook, per-person noise-message tracking, reconciling the notebook and Streamlit parsers, and populating `Final_Submission/` for hand-off.